

Appendix

Perception Entropy for Path Planning in Autonomous Rover Navigation

Christian Medeiros Adriano, Mostafa Wael Aboalfotoh[†], Sona Ghahremani, Holger Giese
Hasso Plattner Institute, University of Potsdam, Germany, German International University, Berlin, Germany[†]
Email: {firstname.lastname}@hpi.de, mostafa.aboalfotoh@student.giu-berlin.de[†]

A. Dataset

We use a subset of the ROVER dataset [1], a multi-season outdoor visual SLAM benchmark collected with a small-scale UGV equipped with an Intel RealSense D435i RGB-D camera, where the calibrated and synchronized sensor suite provides millimetre-accurate ground-truth poses in TUM format aligned to a global frame. For our experiments, we use the ROVER_data/realsense_D435i/sequence, which contains paired color images (rgb/) and 16-bit PNG depth maps (depth/), converting raw depth values to metres with a scale factor of 10^{-3} and associating RGB-depth pairs via timestamps from rgb.txt and depth.txt with a maximum 30 ms difference to ensure temporal alignment. Ground-truth camera poses are stored in groundtruth.txt in the standard TUM RGB-D pose representation $[x, y, z, q_x, q_y, q_z, q_w]$, and we follow the ROVER protocol by discarding frames 1–39 to allow pose convergence, using frames 40–230 as our training window and selecting the pose at frame 60 as the navigation goal for downstream planning and control tasks.

B. Occupancy Grid Construction and Fog-of-War Setup

We construct a 2-D occupancy grid on the ground plane using the (x, y) components of the ground-truth robot poses from frames [40, 230]. The grid domain is defined by computing an axis-aligned bounding box over all valid poses in this range and expanding it by 1.0 m on each side to cover the full navigable area, discretised at resolution $r = 0.05$ m.

For each frame i , the 16-bit depth image is back-projected into 3-D points in the camera frame using pinhole intrinsics ($f_x = f_y = 610$ px, $c_x = 320$ px, $c_y = 240$ px), clipped to the range $[0.5, 10.0]$ m to reject sensor noise. These points are transformed into world coordinates using the ground-truth position (x_i, y_i) and heading $\psi_i = \text{atan2}(\Delta y_i, \Delta x_i)$ derived from consecutive pose differences. World points are then projected onto the 2-D grid: a cell is marked *occupied* ($= 1$) if the fraction of points within the obstacle height band $[0.8 \text{ m}, 2.5 \text{ m}]$ exceeds a hit-ratio threshold of 0.2 (with a minimum of 3 depth hits per cell); otherwise, it is marked *free* ($= 0$), or remains *unknown* (-1) if unobserved.

Observations from all frames are fused via confidence-weighted accumulation with per-cell confidence $\tanh(\text{ray count}/20)$; a new observation overwrites an

existing one only if it has strictly higher confidence, and high-confidence free space (confidence ≥ 0.3) can override a previous obstacle label. The resulting global occupancy grid ("god-map") is precomputed once, kept fixed during training as a hidden oracle for fog-of-war simulation, and at each time step the RL agent observes only a local, ego-centric crop of this grid with fog-of-war applied, yielding a compact top-down view of nearby free space and obstacles.

C. Active Perception

The agent operates under a fog-of-war constraint: the local map $\mathcal{M}$ is initialised as entirely unknown (-1 everywhere) and is revealed incrementally through scanning. Prior to training, a single 60° forward cone raycast is performed at the agent's starting position (facing North), revealing cells from the oracle map into $\mathcal{M}$. During an episode, a scan is triggered before each step if: $d_{\text{dir}}(s, a) < \tau_{\text{scan}}$ and the ray in direction a terminates at an unknown cell ($\text{stop_type}(s, a) = -1$), where $\tau_{\text{scan}} \in \{5, 10, 15, 20\}$ cells is the scan depth threshold, $d_{\text{dir}}(s, a)$ is the directional distance to the nearest unknown cell ahead, and s is the current cell. Each triggered scan reveals a 60° forward cone of radius $r_{\text{scan}} = \rho_{\text{perc}} + \tau_{\text{scan}} \cdot r$ metres from the oracle map, where ρ_{perc} is the perception radius in metres and r is the grid resolution in metres per cell. After each scan, the risk tensor and environment map are updated with the newly revealed cells. The scan history resets at every episode, allowing episodes to trigger scans at any position.

D. Choice of RL Training Methods

We use tabular SARSA (on-policy) and Q-learning (off-policy) to study how perception uncertainty shapes learning under contrasting risk profiles [2]. Both algorithms share the same safety-aware action scoring layer and learn action-value functions from interaction data, so perception-induced errors enter only through the data distribution rather than the control interface. SARSA is more conservative because it updates values for actions taken under the current ϵ -greedy policy, making it sensitive to large negative rewards, whereas Q-learning is more risk-seeking since it updates toward the greedy action regardless of the exploratory action executed, favoring short, high-payoff paths when rare severe failures exist. Comparing both methods under identical safety-aware training and evaluation conditions lets us attribute differences

in risk-taking directly to their on- vs. off-policy learning dynamics and thus isolate how perception uncertainty interacts with RL control.

E. Evaluation Protocol

For each scan-depth threshold, we train agents and record per-episode cumulative reward, path length, mean TD-error, goal-success flag, average safety margin (mean distance-transform value along the path), and Perception Entropy H_{perc} (Shannon entropy of the state-visitation distribution). Safety-margin differences across thresholds are tested for significance using a two-sided Mann–Whitney U test with $\alpha = 0.05$, and training hyperparameters (Table I). After training, the greedy path is obtained by following $\arg \max_a Q(s, a)$ from the start cell, breaking ties by Euclidean distance to the goal.

TABLE I
RL HYPERPARAMETERS

Parameter	Symbol	Value
Learning rate	α	0.4
Discount factor	γ	0.95
Initial epsilon	ε_0	0.99
Terminal epsilon	$\varepsilon_{\min}$	0.1
Epsilon decay	λ	auto
Episodes per run	N	500
Max steps per episode	$T_{\max}$	400
Goal reward	R_{goal}	1000
Wall penalty	R_{coll}	-20
Progress scale	ω	4
Perception radius	ρ_{perc}	1.0m
Scan depth threshold	τ_{scan}	{5, 10, 15, 20}
View Cone angle	θ	60°

Algorithm 1 Active Perception RL Navigation — Full Training Pipeline

```

1: Input: Oracle map  $\mathcal{M}^*$ , start cell  $s_0$ , goal cell  $G$ 
2: Parameters:  $N_{\text{ep}}=500$ ,  $N_{\text{steps}}=400$ ,  $\alpha=0.4$ ,  $\gamma=0.95$ ,
    $\varepsilon_{\text{start}}=0.99$ ,  $\varepsilon_{\text{end}}=0.10$ ,  $\tau_{\text{scan}}$ ,  $\rho_{\text{perc}}$ ,  $\zeta_{\text{obs}}=10$ ,  $\zeta_{\text{unk}}=1$ 
3: // — Initialization
4:  $Q(s, a) \leftarrow 0 \quad \forall s, a$  Q-table starts at zero — agent has no prior knowledge
5:  $\mathcal{M} \leftarrow \{-1\}^{H \times W}$  agent map: every cell unknown
6:  $\varepsilon \leftarrow \varepsilon_{\text{start}}$ 
7:  $r_{\text{scan}} \leftarrow \rho_{\text{perc}} + \tau_{\text{scan}} \cdot r$  dynamic scan radius ( $r$  = grid resolution in metres)
8: // — Initial scan at  $s_0$  before any training
9:  $\mathcal{M} \leftarrow \text{REVEAL}(\mathcal{M}, \mathcal{M}^*, s_0, 360^\circ, r_{\text{scan}})$ 
10:  $\Xi \leftarrow \text{BUILDRISKSENSOR}(\mathcal{M})$  risk pre-computed for ALL free cells, all 8 directions
11: // — Training loop
12: for episode  $e = 1$  to  $N_{\text{ep}}$  do
13:    $s \leftarrow s_0$ 
14:   for step  $k = 1$  to  $N_{\text{steps}}$  do
15:     // Step A — Action selection via  $\varepsilon$ -greedy
16:     if  $\text{rand}() < \varepsilon$  then
17:        $a \leftarrow \text{UniformRandom}(\text{ValidActions}(s))$  explore: ignore Q-table, pick random direction
18:     else
19:        $a \leftarrow \arg \max_{a'} Q(s, a')$  exploit: pick direction with highest Q-value
20:     end if
21:     // Step B — Scan check in chosen direction  $a$  before moving
22:     if  $d(s, a) < \tau_{\text{scan}}$  and  $\mathcal{M}(c_{\text{stop}}) = -1$  then
23:       // Scan: single direction only, 60° cone
24:        $\mathcal{M} \leftarrow \text{REVEAL}(\mathcal{M}, \mathcal{M}^*, s, 60^\circ, r_{\text{scan}}, \text{dir}=a)$  copies cells from oracle into agent map along direction  $a$  only
25:        $\Xi \leftarrow \text{BUILDRISKSENSOR}(\mathcal{M})$  full rebuild: ALL free cells updated — cascade applies
26:     end if
27:     // Step C — Execute action, compute reward
28:      $s' \leftarrow s + \Delta(a)$ 
29:      $\Xi_t \leftarrow \Xi[s', a]$  directional risk at destination cell for chosen direction
30:      $\Pi_t \leftarrow (d(s, G) - d(s', G)) \cdot w_{\text{prog}}$  progress reward: positive if moved closer to goal
31:      $C_t \leftarrow \text{MovementCost}(a)$  1.0 cardinal, 1.0 diagonal (configurable)
32:      $R_t \leftarrow \begin{cases} +1000 & \text{if } s' = G \\ R_{\text{coll}} & \text{if } s' \in \{\text{obstacle, unknown}\} \\ \Pi_t - C_t - \Xi_t & \text{otherwise} \end{cases}$ 
33:     // Step D — Q-table update for this visited cell only
34:      $Q(s, a) \leftarrow Q(s, a) + \alpha[R_t + \gamma \max_{a'} Q(s', a') - Q(s, a)]$  only  $(s, a)$  updated — all unvisited cells keep stale Q-values
35:      $s \leftarrow s'$ 
36:     if  $s = G$  or  $k = N_{\text{steps}}$  then
37:       break
38:     end if
39:   end for
40:   // Step E — Epsilon decay after each episode (never resets)
41:    $\varepsilon \leftarrow \max(\varepsilon_{\text{end}}, \varepsilon \cdot \varepsilon_{\text{decay}})$  persists across scans and episodes — never reset
42: end for
43: // — Greedy path extraction after all training
44: return  $\text{FollowGreedy}(Q, s_0, G)$   $\varepsilon=0$ : always pick  $\arg \max_a Q(s, a)$ 

```

Algorithm 2 BUILD RISK TENSOR — Directional Risk for All Free Cells

```

1: Input: Accumulated agent map  $\mathcal{M}$  of shape  $H \times W$ 
2: Output: Risk tensor  $\Xi$  of shape  $H \times W \times 8$ 
3:  $\Xi \leftarrow \mathbf{0}^{H \times W \times 8}$ 
4: for each free cell  $(r, c)$  where  $\mathcal{M}(r, c) = 0$  do
5:   for each direction  $i \in \{0, \dots, 7\}$  do
6:     Cast ray  $(r, c) \rightarrow i$ ; record distance  $d$  to nearest blocker
       and stop cell  $c_{\text{stop}}$ 
7:     if  $\mathcal{M}(c_{\text{stop}}) = 1$  then
8:        $\zeta \leftarrow \zeta_{\text{obs}}$  ray hit obstacle — high hazard
9:     else if  $\mathcal{M}(c_{\text{stop}}) = -1$  then
10:       $\zeta \leftarrow \zeta_{\text{unk}}$  ray hit unknown terrain — moderate hazard
11:    else
12:       $\zeta \leftarrow 1$  ray reached map boundary — no extra hazard
13:    end if
14:     $V_\varphi \leftarrow$  fraction of unknown cells inside  $60^\circ$  cone at  $(r, c)$ 
       facing  $i$  invisible rate: 0.0 = fully mapped, 1.0 = fully unknown
15:     $\Xi[r, c, i] \leftarrow \zeta \cdot \underbrace{\frac{d_{\text{var}}}{d + d_{\text{var}}}}_{\varphi^{(d)}} \cdot V_\varphi$  close blocker  $\Rightarrow$  high  $\varphi$ ;
unmapped cone  $\Rightarrow$  high  $V_\varphi$ ; both together  $\Rightarrow$  maximum risk
16:  end for
17: end for
18: Cascade note: when a scan reveals new cells, rays from existing
   free
19: cells whose path previously ended at unknown ( $-1$ ) may now
   travel further
20: and terminate at a different  $c_{\text{stop}}$  — changing  $\zeta$ ,  $d$ , and  $V_\varphi$ 
21: for those cells. No cell moved; their risk values changed because
22: the map around them changed. This is why the full tensor is
   rebuilt
23: every scan, not just the newly revealed cells.
24: return  $\Xi$ 

```

REFERENCES

- [1] F. Schmidt, J. Daubermann, M. Mitschke, C. Blessing, S. Meyer, M. Enzweiler, and A. Valada. Rover: A multiseason dataset for visual slam. *IEEE Transactions on Robotics*, 41:4005–4022, 2025.
- [2] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. Adaptive Computation and Machine Learning. MIT Press, Cambridge, Massachusetts, 2 edition, 2018.